DEVSECOPS TECHNICAL ASSESSMENT · TRACK 1

Kubernetes & container security: review, target state and go-live plan

Srikar Adusumilli · October 2026

Repository: github.com/adusumillisrikar/k8s-container-security-assessment

TODAY

One recommendation, two decisions

The recommendation

Fix 10 go-live blockers (about 12 engineer-days) and put in **four platform controls** that close most of the remaining risk for every workload at once.

Decision 1: COTS product

Accept the vendor container at **Medium** residual risk with compensating controls, signed by the system owner, endorsed by the ITSO, for 6 months with a funded exit plan.

Decision 2: data residency

Does the data classification allow a second-region DR copy? It changes the regional recovery answer from "accepted risk" to "about 8 hours".

Agenda

1. Where we stand and why

2. The design as one story

3. Architecture and the COTS decision

4. The first 90 days

EXECUTIVE SUMMARY

Not ready for go-live today. Ready within the window: blockers first, then four platform controls.

- **Too much power in the wrong places.** A build pipeline account holds the cluster's master key, and some apps control the servers they run on.
- **Nothing checks what goes in.** No gate stops a dangerous configuration, and no check that what runs is what we built.
- **Nothing separates apps from each other.** Every app can reach every other app; passwords sit where many can read them.
- **We wouldn't know.** Audit logging is off and there's no runtime detection.

31 findings



10 go-live blockers, about **12 engineer-days** to close.

Most of the rest close with **4 platform controls** rather than app-by-app work.

HOW FINDINGS WERE PRIORITISED

Blast radius × who can reach it, then: what has to happen before go-live?

IMPACT	MEANING
4	Whole cluster, a node and its cloud identity, the control plane
3	Many workloads, a namespace, or credentials usable elsewhere
2	One workload
1	Hygiene

EXPLOITABILITY	WHO CAN REACH IT
4	Anyone on the internet
3	Any pod, anyone who can run a CI job or pull an image
2	Needs a specific foothold

> Why not just CVSS? Most findings are **misconfigurations**, not CVEs. A privileged pod is a nuisance in a sandbox and a cluster takeover on a shared node.

FIX WINDOW	MEANING
P0	Blocks go-live. No risk acceptance offered.
P1	Fix, or go live with a compensating control and a signed acceptance
P2 / P3	Owned backlog, reviewed monthly

TOP FINDINGS

Each of these, on its own, hands over the cluster

F-01

CI account is cluster-admin

The audit-finding pattern. Anyone who can run a pipeline holds the master key.

F-02

Root on the node + open web shell

Privileged pod with the host disk mounted and a terminal with no password.

F-03

A text box that runs commands

Command injection in a privileged pod: one form field to root on the server.

F-04

One chart install = admin for all

Grants `*/*/*` to the default identity most pods use.

Why the order matters: findings combine into chains

A · CI to cluster
Pipeline (F-01) + no audit log (F-16) = full control, no trace.

B · Any pod to node to cluster
Flat network (F-12) → web shell (F-02) → node → tokens of every pod on it.

C · Internet to cloud keys
Exposed debug app (F-10) + SSRF (F-09) → node's cloud credentials (F-19).

TESTED, NOT JUST DESIGNED

Would this design have caught the audit finding before merge?

RUN AGAINST KUBERNETES-GOAT'S REAL CLUSTERROLEBINDING	RESULT	WHAT IT MEANS
Our Conftest policy (pre-merge gate)	**Blocked**	`[F-01] ClusterRoleBinding/superadmin binds cluster-admin`
Our Kyverno policy (admission)	**Blocked**	Refused at the API server, however it arrives
Checkov (generic catalogue)	**Not flagged**	Its RBAC checks read role contents; `cluster-admin` is built in
Trivy (generic catalogue)	**Medium**	Passes a typical "fail on High/Critical" gate

Generic scanners are necessary but not sufficient. The rule written for **our** incident is the one that stops it. Same for code: three org Semgrep rules catch the injection and SSRF the registry packs missed.

Testing also found a **bug in my own policy** (silently skipped in background mode) and **two findings I'd missed**. Policies get tested like code.

Scanners run pinned by digest, with no network and read-only inputs, after the 2026 Trivy/KICS image compromises. Evidence: 01-findings/evidence/

THE DESIGN AS ONE STORY

Prevent what we can, contain what gets through, detect the rest

1 · Prevent

Pipeline gates + admission control. Signed images only. **CI holds no cluster credentials**: the cluster pulls approved changes from Git.

→

2 · Contain

Default-deny network. One cloud identity per workload. Secrets fetched at runtime, never in Git or images.

→

3 · Detect & respond

Audit log + runtime detection to a write-once archive. Contain without destroying evidence.

Same chain B, broken three times

ATTACKER STEP	STOPPED BY
Deploy a privileged pod with the host disk mounted	**Prevent:** refused at merge (Conftest) and at the API server (Pod Security)
From a compromised pod, reach another app's web shell	**Contain:** default-deny NetworkPolicy drops the connection
Read a service-account token or call the node's metadata service	**Detect:** Falco rules alert; token isn't mounted; IMDS hop limit blocks the call

Hard-fail what gives an attacker a path; warn, track and ratchet the rest

GATE	MODE
Secrets (gitleaks)	**Hard**
Org policy: cluster-admin, privileged, hostPath, unpinned images (Conftest)	**Hard**
IaC: public endpoint, logging, encryption (curated Checkov list)	**Hard**
Code: shell injection, SSRF, debug mode (Semgrep + org rules)	**Hard**
Fixable Critical CVEs, any secret in an image (Trivy)	**Hard**
runAsNonRoot, read-only FS, High CVEs	**Warn → hard in 60 days**
Reliability hints (kube-score)	Advisory

- **Sign** every image with a KMS-held key; **verify** at admission.
- **SBOM** attached to the image and re-checked daily for new CVEs.
- **SLSA Build L2 now**, L3 after runner isolation work. Claiming L3 on shared runners would be paperwork.
- **Production:** security + service-owner approval; nobody approves their own deploy.
- **Exceptions:** in Git, owned, max 90 days, pipeline fails when they expire.
- **Speed budget:** merge-request checks under 5 minutes, all in parallel. A slow gate gets bypassed.

> Other clouds: policies, scanners and Argo CD are unchanged. Registry, signing key and identity trust change.

Deny by default, allow by request. No service mesh yet, on purpose.

- **Turn on enforcement** in the existing VPC CNI; standard NetworkPolicy only, so it ports to AKS and GKE as-is.
- **Default-deny** in and out per namespace, rolled out from two weeks of flow logs.
- **No internet by default.** Allowed domains go through an egress firewall.
- **Metadata service unreachable** from pods (IMDSv2, hop limit 1).
- **One cloud identity per workload** (Pod Identity); the node role holds almost nothing.

Service mesh: not for go-live

For: mTLS everywhere, L7 authorisation, uniform across clouds.

Against, now: it lands on a stretched team; sidecars are a risk for the COTS image; none of the Critical findings is fixed by a mesh.

Revisit in 2 quarters if: the standard requires in-cluster mTLS, service-to-service calls outgrow IP rules, or AKS/GKE need one story. Then Istio ambient (no sidecars).

When prevention fails: see it, contain it, keep the evidence

Detection

- **Falco** on every node: same rules on EKS, AKS, GKE.
- **Audit log** rules: new cluster-admin binding, secret listing.
- **GuardDuty** as a second opinion on AWS.
- Only Critical pages; two-week baseline before anything pages; every paging rule has a runbook.

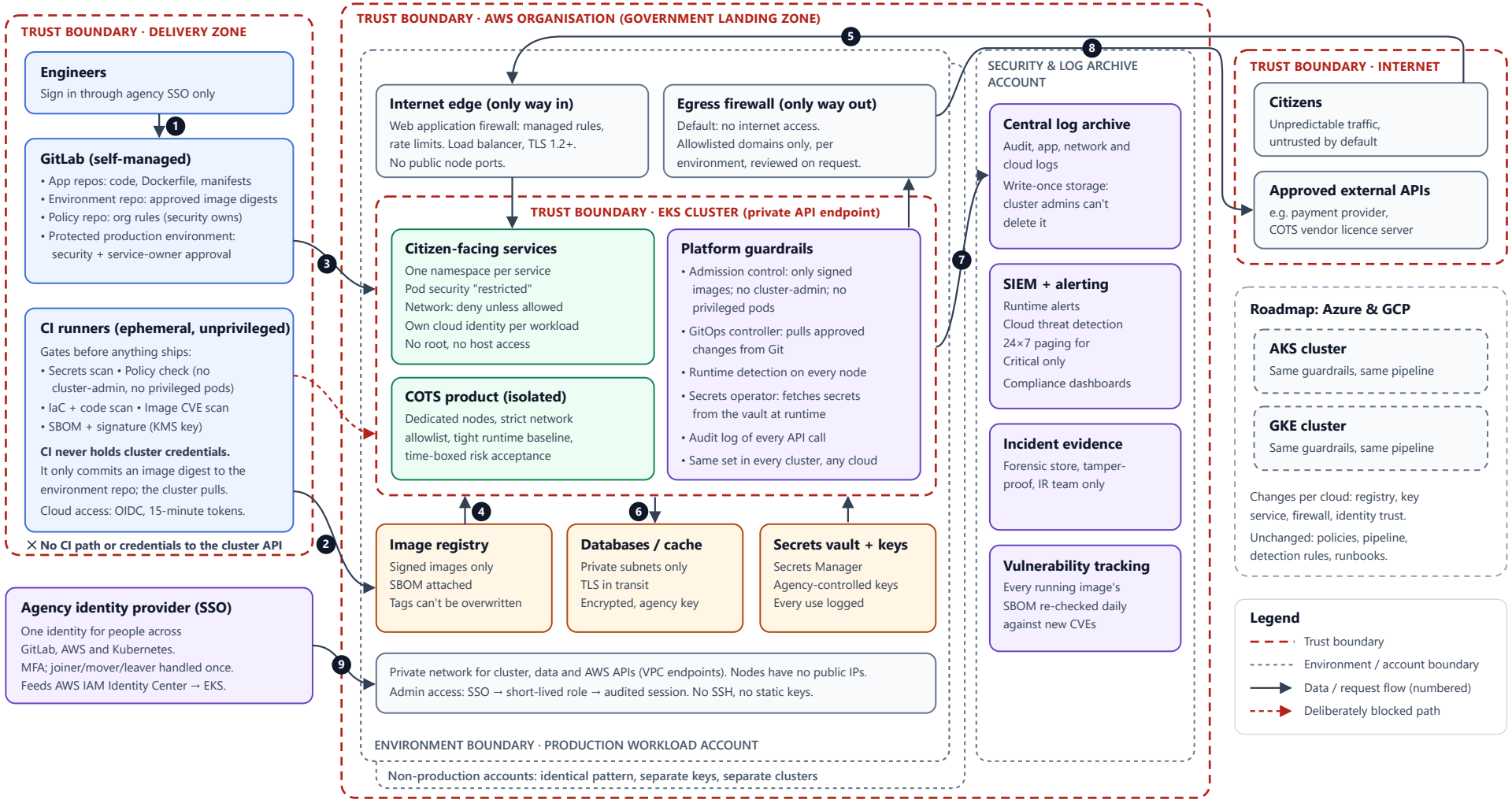
Scenario: confirmed container escape attempt

1. **Capture first** (memory checkpoint, filesystem diff, logs), from outside the container
2. **Relabel the pod**: it leaves the Service, a clean replica starts, and no network policy allows it anything
3. **Cordon, don't drain** the node; swap it to a forensic security group
4. **Revoke and rotate** every credential the pod and node could reach
5. **Replace, don't clean**: rebuild from Git at signed digests

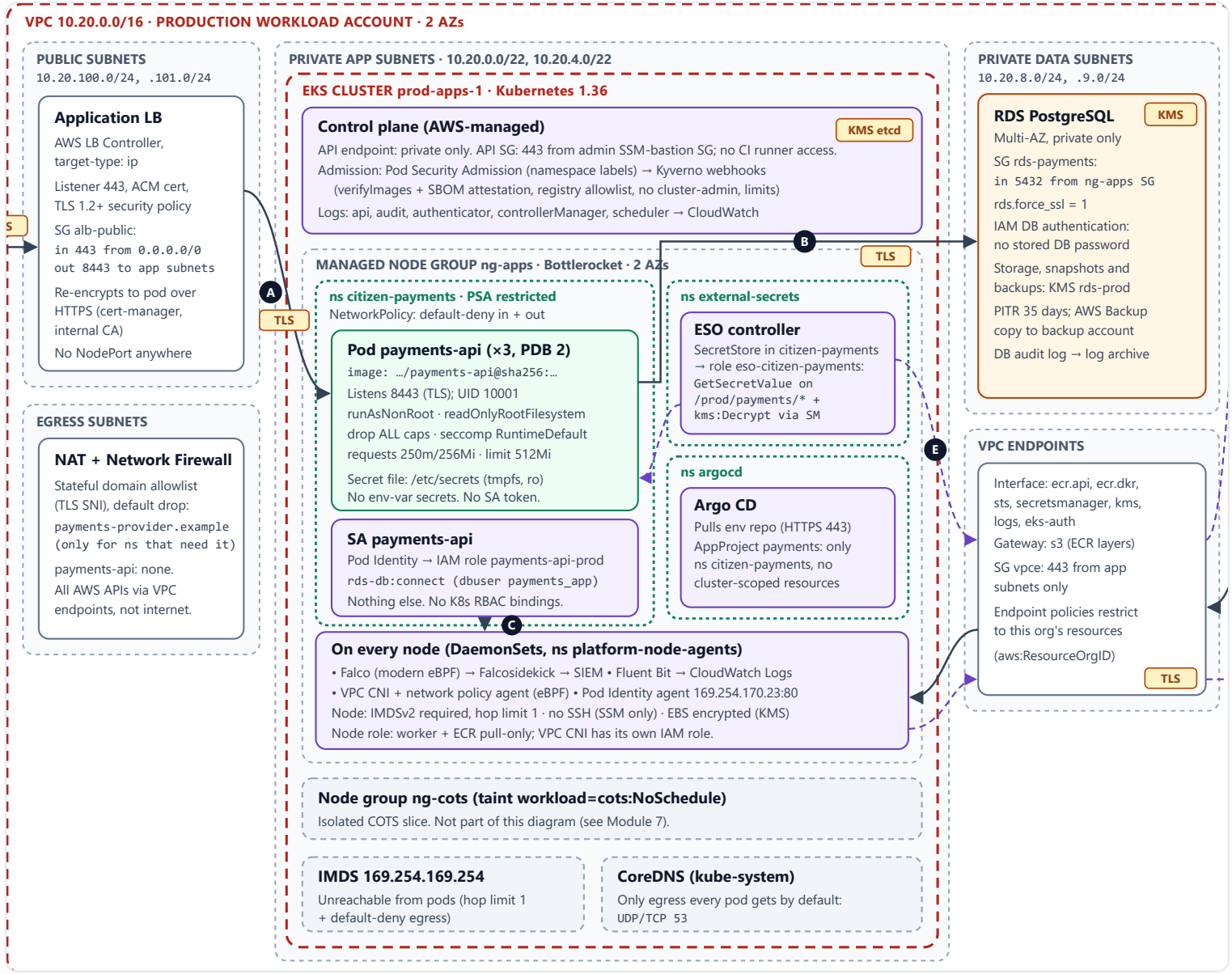
> ITSO during the incident: fixed-cadence updates with "decisions needed from you". Afterwards: root cause, impact evidence, dated actions.

Target-state secure Kubernetes platform: high-level design

System context for governance review. Numbered flows are explained in architecture-narrative.md. Production shown; non-production accounts follow the same pattern.



LOW-LEVEL DESIGN: ONE CITIZEN REQUEST, END TO END (FULL DIAGRAM: BACKUP A5)



#	STEP	CONTROL
1	Citizen → WAF → ALB	443, TLS 1.2+
A	ALB → pod	8443, TLS again; only from ALB subnets
C	Pod → Pod Identity agent	Short-lived creds, one permission: `rds-db:connect`
B	Pod → RDS	5432, TLS forced, IAM token, no password
E	ESO → Secrets Manager	Own namespace's path only; mounted as a file
F	CI → ECR	Push only; no route to the cluster
✕	Anything else	Dropped: IMDS, internet, other namespaces

Pod: non-root, read-only filesystem, no capabilities, no SA token. Falco and Fluent Bit on every node; admission checks the signature before it runs.

DECISION 1 · COTS PRODUCT

Go live with the vendor image at Medium residual risk, for 6 months

Assumed gap: end-of-life base OS with unfixable CVEs, runs as root, needs a writable filesystem. Vendor won't rebuild without a paid contract.

Compensating controls (reused, not invented)

- Narrow admission exception: only **this exact digest**, still no privileged or host access
- Dedicated nodes, 3-flow network allowlist, one licence domain
- Strict runtime baseline; auto-quarantine on drop-and-execute
- Own secrets store and DB user; daily KEV check reopens the decision

	RATING
Inherent	High
Residual, with controls	Medium

Not covered: an RCE in the product itself would still expose its own data. The controls limit where an attacker goes next and make sure we see it.

Ask: system owner accepts, ITSO endorses. 6 months, quarterly review, funded exit: support contract, SaaS, our own rebuild, or replace.

DECISION 2 · RESILIENCE

A rebuilt cluster in under 2 hours, from Git. Region loss is a residency question.

PAYMENTS-API: FAILURE	RPO	RTO
Pod / node	0	< 1 min
One availability zone	0	< 5 min
Cluster loss, including compromise	≤ 5 min	≤ 2 h
Data corruption	≤ 5 min	≤ 4 h
Whole region	**Your decision →**	

- Data lives **outside** the cluster, so the cluster is disposable.
- Everything else rebuilds from Terraform + Git; tested quarterly.
- Backups write-once, in a separate account.

> A second AWS region can mean citizen data leaves the country. **If allowed:** about 8 h RTO. **If not:** a signed, documented accepted risk.

THE FIRST 90 DAYS

Blockers first, platform next, then prove it works

Weeks 1–2 · Blockers

- Confirm these findings against the **real** estate, with the delivery teams
- Remove the cluster-admin binding; scope CI to its namespaces
- Remove privileged and host-access pods; rotate exposed secrets
- Turn on audit logging; Pod Security in warn mode everywhere

Weeks 3–6 · Platform, then go-live

- Policy gates and admission: audit → enforce
- Default-deny networking, namespace by namespace
- Pod Identity, IMDSv2; secrets into Secrets Manager
- Image signing; COTS controls and signed acceptance

Weeks 7–12 · Prove and harden

- GitOps cut-over: CI loses cluster access entirely
- Falco paging on; soft gates ratchet to hard
- First cluster rebuild drill and DB restore test
- Run the same policies against a test AKS/GKE cluster

How I'd measure it: blockers closed with evidence, % of namespaces at default-deny and Pod Security `restricted`, time to detect a test binding, and measured rebuild time against the 2-hour target.

SUMMARY

What I'm asking for

Agree the plan

10 blockers before go-live, then the four platform controls, in the 90-day order shown.

Decide on the COTS product

Medium residual risk, accepted by the system owner and endorsed by the ITSO, for 6 months.

Decide on residency

Second-region DR copy allowed for payments data, or regional loss accepted and documented.

Questions

github.com/adusumillisrikar/k8s-container-security-assessment

Backup slides: A1 shared responsibility · A2 one finding, two audiences · A3 how this was verified · A4 secrets options · A5 full LLD

BACKUP A1 · SHARED RESPONSIBILITY

The provider runs the control plane. Everything that decides who gets in, what runs and what talks is ours.

	EKS	AKS	GKE
Audit logs	Off until enabled (F-16)	Not collected until a diagnostic setting routes them	Admin Activity always on; Data Access off by default
Worker node OS	AWS publishes AMIs, we roll them out (Auto Mode / Fargate: AWS)	Node image auto-upgrade channel	Auto-upgrade by default; Autopilot: Google runs nodes
Pod hardening default	None: ours to enforce	None: ours to enforce	Autopilot blocks privileged pods
Secrets at rest	AWS-owned key by default; our key optional (F-23)	Platform-encrypted; our KMS key optional	Encrypted by default; Cloud KMS layer optional
API endpoint	Public by default (F-15)	Public by default	External endpoint by default
NetworkPolicy	Turn on in VPC CNI (F-12)	Engine chosen at cluster creation	Enforced by Dataplane V2
Control-plane SLA	Included	Standard/Premium tier only	Included

> **Design rule:** build to the weakest default on every row, so the baseline holds on any cloud. Our own Pod Security, audit logs shipped explicitly, our keys, private endpoints, AKS Standard tier.

BACKUP A2 · ONE FINDING, TWO AUDIENCES

F-01 for the governance board, and for the delivery engineer

To the board: how bad, and what do you need from us?

- Our build system holds the master key to the cluster. A tampered build could control every service and hide its tracks.
- **Blocks go-live.** About one engineer-day to fix. No risk acceptance offered.
- You'll see evidence that both the pipeline and the cluster refuse it.

To the engineer: what changes, and how do I know it works?

- CI can now deploy Deployments, Services, ConfigMaps and Ingresses in **your** namespaces only. No Secrets, no `exec`, nothing cluster-wide.
- A deploy fails with `forbidden`? The error names the permission; request just that one in the MR.
- Check (as tested): `kubectl auth can-i create clusterrolebindings --as=ci --as-group=ci-deploy:<your-ns>` → `no`

> Same facts, different questions. The worked engineer version for the top three findings is the remediation pack (Module 6).

Testing changed the submission, which is why it was worth doing

FOUND BY TESTING OR REVIEW	FIXED
Kyverno cluster-admin rule silently skipped (background mode)	Admission-only; re-tested
Wrong Checkov check ID in the pipeline	Corrected; Conftest covers F-01
Semgrep claim overstated: registry packs missed injection and SSRF	3 org rules; pass on fixed code
Two findings the manual review missed	Added as F-30, F-31
Stale facts: K8s 1.33, EKS secret encryption default, auth defaults	1.36; F-23 and F-15 reworded
Final review: a pipeline image with no shell; EKS's own admin binding blocked; quarantine still allowed DNS	All fixed and re-tested

- Conftest, Kyverno, Checkov, Trivy, kube-score, Semgrep, gitleaks, kubeconform, Falco, kube-bench
- Live checks on a throwaway kind cluster: Pod Security blocks goat's pods; the fixed ones pass; RBAC before and after
- Every tool pinned by digest, two-week release cooldown, offline where possible. No goat image was ever pulled or run
- This repo's full history: no secrets (gitleaks)

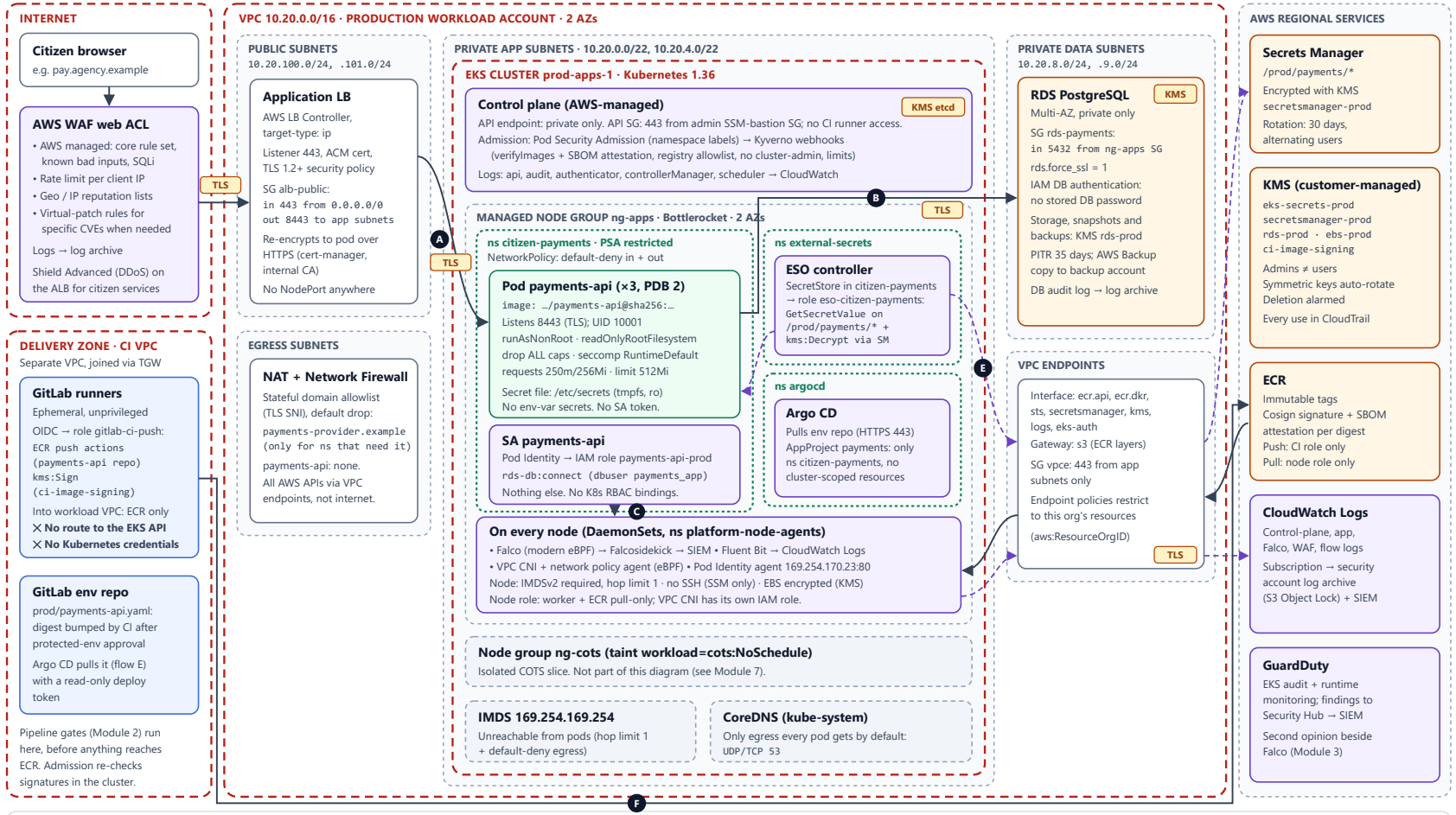
External Secrets Operator now, with the backend kept swappable

	ESO + SECRETS MANAGER	VAULT / OPENBAO	SEALED SECRETS / SOPS
Run cost	Low: managed backend	High: a tier-0 service to run	Low
Rotation	Managed, dual-user; no manual restarts	Best: short-lived dynamic credentials	Manual re-encrypt and commit
Read audit	CloudTrail on every read	Detailed audit log	Git shows changes, not reads
Multi-cloud	Same API; backend per cloud	One backend everywhere	Per-cloud KMS or per-cluster keys
Ready for go-live	Weeks	Months	Weeks, but weak on rotation

Decision point at AKS/GKE onboarding: if secrets must be shared across clouds or dynamic credentials are needed, repoint the per-namespace SecretStores at Vault. App manifests don't change. Best secret of all: none (IAM database auth).

Low-level design: citizen request → payments-api → database (production)

One representative slice of the HLD, at implementation detail. Letters on flows match the NetworkPolicy table. TLS / KMS badges show where encryption happens.



Flow	From → To	Port / protocol	Allowed by	Encryption
A	ALB subnets 10.20.100.0/23 → payments-api pods	TCP 8443	NP payments-api-ingress-alb + SG alb-public	TLS (internal CA cert via cert-manager)
B	payments-api → RDS 10.20.8.0/23	TCP 5432	NP payments-api-egress-db + SG rds-payments	TLS enforced (rds.force_ssl), IAM auth token
C	payments-api → Pod Identity agent 169.254.170.23	TCP 80 (node-local)	NP allow-pod-identity-agent	Node-local link; returns short-lived creds
D	all pods in ns → CoreDNS (kube-system)	UDP/TCP 53	NP allow-dns (namespace template)	n/a (in-cluster DNS)
E	ESO, nodes, Argo CD → VPC endpoints / GitLab (TGW)	TCP 443	NP in platform namespaces only	TLS
F	GitLab runners (CI VPC) → TGW → ECR endpoint → ECR (push)	TCP 443	TGW route table + SG vpce; ECR repo policy	TLS; image signed with KMS key
✕	payments-api → anything else (IMDS, internet, other namespaces)	any	dropped by default-deny-all	